Introduction to GPU Programming

Wim R.M. Cardoen

Center of High-Performance Computing (CHPC)
University of Utah

November 3, 2025

Outline

Motivation

Hardware

- Streaming multiprocessor (SM)
- Warps

Software

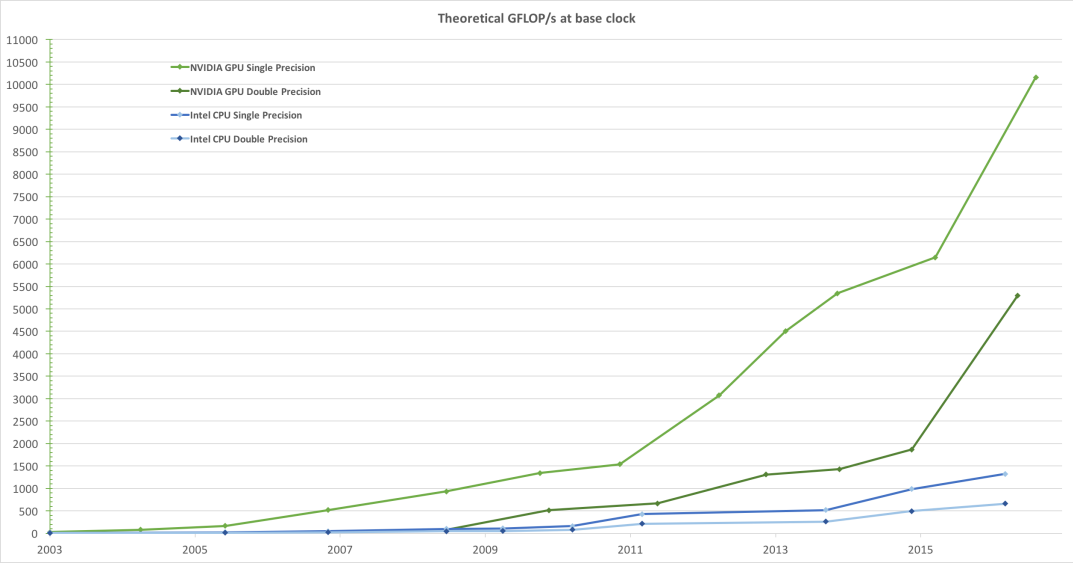
- GPGPU & CUDA
- Structure of a GPU computation
- Case study: matrix multiplication
- Building CUDA applications & useful env. variables
- Profiling & debugging
- Important CUDA libraries
- Alternatives to CUDA
- Links

Use of GPUs at the CHPC

- GPUs available at CHPC
- How to access the GPUs at CHPC

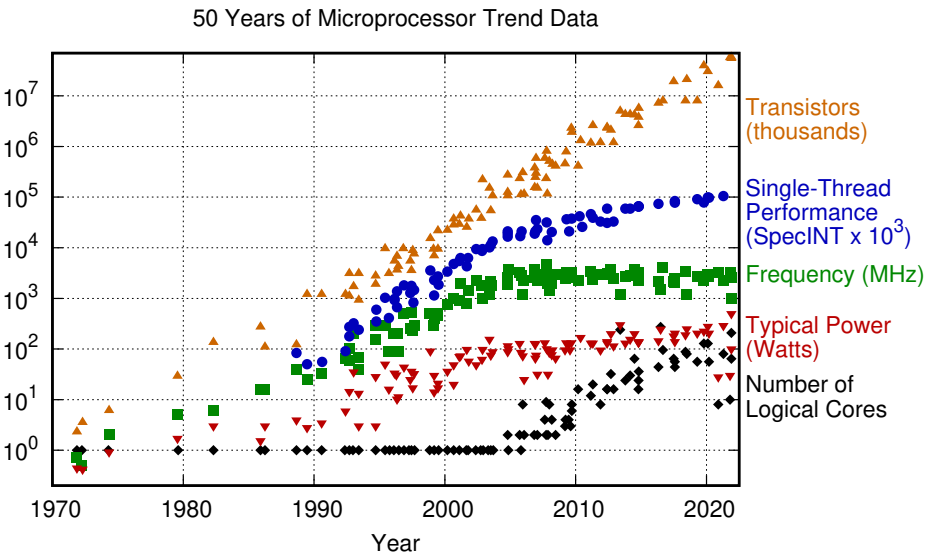
Questions

Theoretical GFLOP/s: GPU vs. CPU



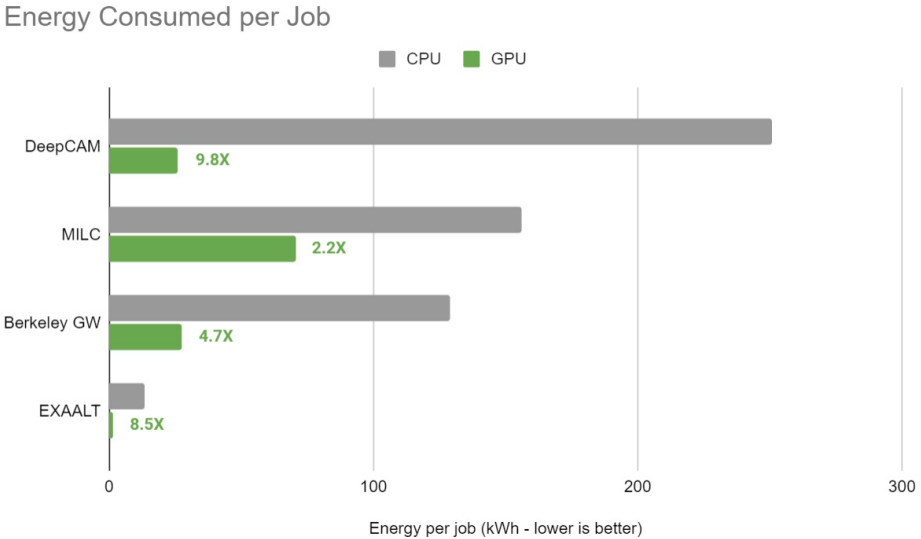
Theoretical GFLOP/s: GPUs vs. CPUs. Source: NVIDIA CUDA C Programming Guide (2018).

CPU processor trend (last 50 years)



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Energy efficiency per job: GPU vs. CPU



Energy efficiency per job (NERSC, May 2023). Source: https://blogs.nvidia.com/blog/gpu-energy-efficiency-nersc/

Streaming Multiprocessor (SM)

- GPU device connected to the CPU by a PCIe bus.
- Each GPU device contains an array (**x**) of Streaming Multiprocessors **(SM)**.
- Each SM has:
  - a Single-Instruction Multiple-Thread (`SIMT`) Architecture.
  - **y** regular cores and [**z** tensor cores].
- Scalable: newer generations increase **x**, **y** and [**z**], e.g.:
  - `NVIDIA A100-PCIE-40GB` (*notch293*)
    - Global memory: 40 GB.
    - 108 SMs, 64 Cores/SM, 4 Tensor Cores/SM.
    - GPU Max. Clock Rate: 1.41 GHz.
  - `NVIDIA H100 SXM5 NVL` (*grn008*)
    - Global memory: 93 GB.
    - 132 SMs, 128 Cores/SM, 4 Tensor Cores/SM.
    - GPU Max. Clock Rate: 1.78 GHz.

NVIDIA GH100 SM

NVIDIA GH100 Full Device

NVIDIA GH100 Full Device (144 SMs).

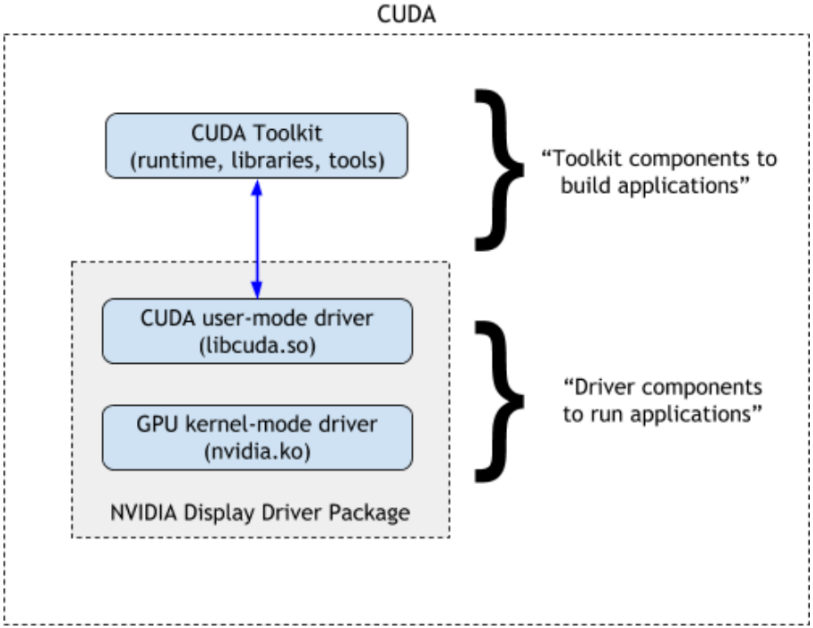
GPU Threads — Warps

- Each SM:
  - generates, schedules, and executes threads in batches of 32 threads.
  - **WARP**: a batch of 32 threads.
- Each thread in a WARP executes the same instructions but runs its own "path".
- If threads within a WARP diverge, the threads become inactive/disabled.

GPGPU & CUDA

- GPU (Graphic Processing Unit): originally developed for graphical applications.
- GP-GPU: General-Purpose GPU, i.e. the use of GPUs beyond graphical applications. **CAVEAT**: problem must be reformulated in terms of the graphics API.
- **2007**: NVIDIA introduces the **CUDA** framework (**C**ompute **U**nified **D**evice **A**rchitecture). The CUDA Toolkit consists of the CUDA Driver and the CUDA Toolkit (`nvcc`, `nvprof`, ..., libraries, header files).
- CUDA API: extension of the `C` language.
- Handles GPU thread level parallelism.
- Deals with moving data between CPU and GPU.
- Also supports C++, `Fortran` and `Python`.

Schema of CUDA Components



Schema of the CUDA Components

Structure of a GPU computation

1. **Allocate** memory space on the GPU device.
2. **Transfer** the data from the CPU to the GPU device.
3. Perform the **calculation** on the GPU device: **kernel** (function executed on the GPU). To enhance performance: keep data as long as possible on the GPU device.
4. **Transfer** the result back from the GPU device to the CPU.
5. **Deallocate** memory space on the GPU device.

Note: source code & makefile available in `./src`

Alloc. & free of global memory on the GPU

- `cudaError_t` — CUDA Error types.
- `cudaError_t cudaMalloc(void **devPtr, size_t size)`
  Allocates memory on the device.
- `cudaError_t cudaFree(void *devPtr)`
  Frees memory on the device.

Note: full example in `./latexinc/ex1.cu`

Copy data between host (CPU) and device (GPU)

`cudaMemcpy` signature:

- ```
  cudaError_t cudaMemcpy(
      void *dst, const void *src,
      size_t count, cudaMemcpyKind kind)
  ```

Note: full example in `./latexinc/memcpy.cu`
  ```

Copy data — direction (`kind`)

- `cudaMemcpyHostToHost`
- `cudaMemcpyHostToDevice`
- `cudaMemcpyDeviceToHost`
- `cudaMemcpyDeviceToDevice`

Note: full example in `./latexinc/ex2.cu`

CUDA Kernel — Declaration

- **CUDA kernel**: function which may run on a GPU device.
- **Kernel declaration** syntax:

  funcspec `void` *kernelName*(*args*){ *body* }
- *funcspec*: `__global__`, `__host__`, or `__device__`
- *kernelName*: name of the kernel/CUDA function.
- *args*: argument list of the kernel/CUDA function.
- *body*: body of the kernel/CUDA function (your code).

CUDA Kernel — Call syntax

- **Kernel call** syntax:

  kernelName<<<*gridSize*,*blockSize*>>>(*args*)
- *gridSize*: size of the grid of thread blocks.
- *blockSize*: size of a thread block.

Function type qualifiers

Qualifier	Called from	Executed on
`__global__`	host	device
`__host__`	host	host
`__device__`	device	device

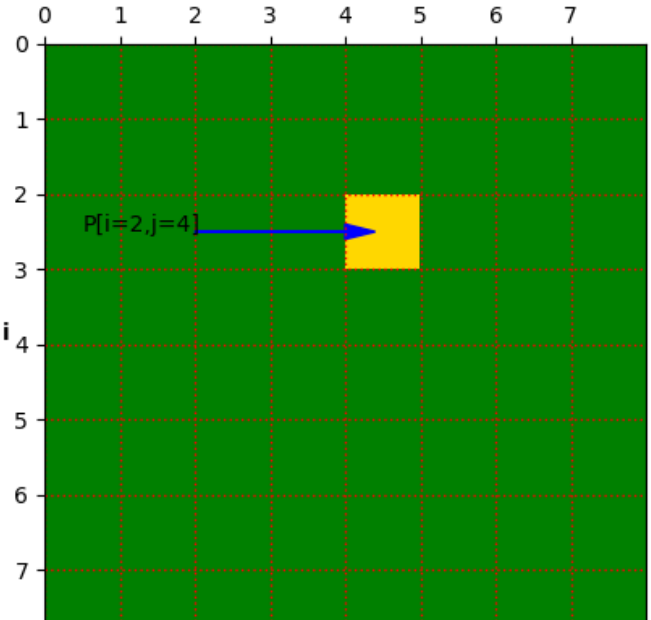
Note: You can have two different versions of a function: one with `__host__` & one with `__device__`.

Grids, Blocks and Threads

We have a hierarchical (software) implementation.

- `uint3`, `dim3`: CUDA structures of unsigned integers `x`, `y`, `z`. `dim3` is based on `uint3` but initializes unspecified components to 1.
- **Grid**: each Grid consists of Blocks. `dim3 gridDim`: dimensions of the Grid. `uint3 blockIdx`: block index within the Grid.
- **Block**: each Block consists of Threads. `dim3 blockDim`: dimensions of the Block. `uint3 threadIdx`: thread index within the Block.

Preliminary notion: matrix storage as a 1D vector



Matrix Multiplication $P = M\,N$

Matrix multiplication: $P = M\,N$ where $M, N \in \mathbb{R}^{d \times d}$

- $P[i, j] = \sum_{k=0}^{d-1} M[i, k]\,N[k, j]$
- $M[i, k]$ is stored as: $M[i * d + k]$
- $N[k, j]$ is stored as: $N[k * d + j]$
- Therefore,

$$P[i * d + j] = \sum_{k=0}^{d-1} M[i * d + k]\,N[k * d + j] \tag{1}$$

Matrix Mul.: kernel (v. 1)

- Each Thread (`threadIdx`) is a 2D object: (`threadIdx.x`, `threadIdx.y`) (cfr. a point in plane geometry).
- Each Thread calculates only 1 element of P.

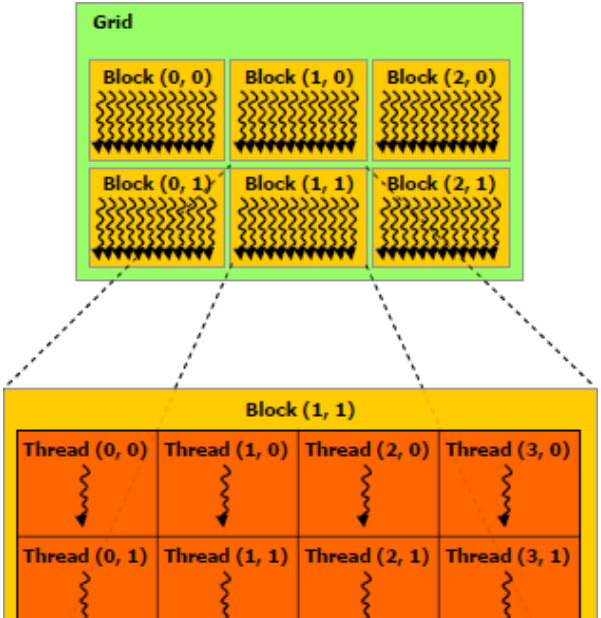
Implementation of Eq. (1) using 1 Block of Threads.
Note: see `./latexinc/matmul/1/mul.cu`

Invoking kernel (v. 1)

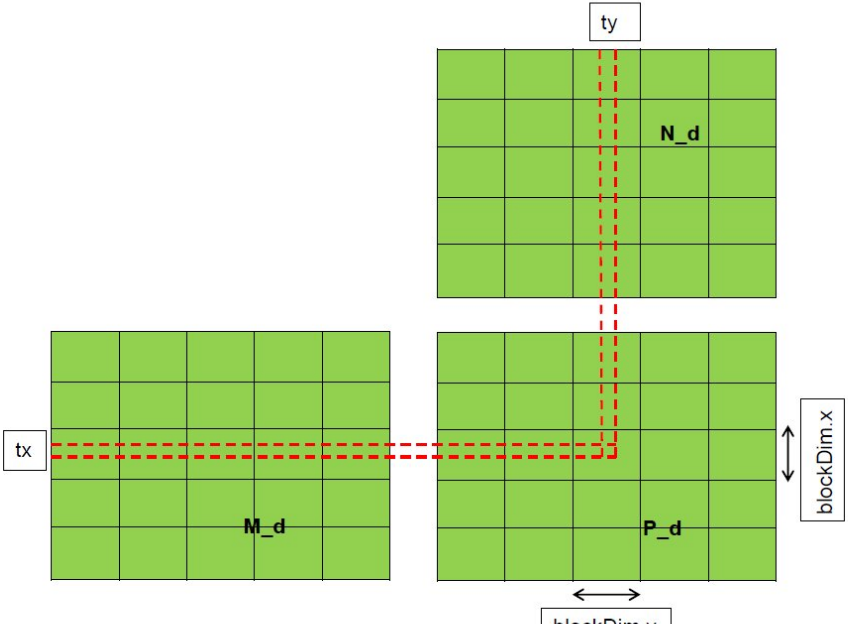
- Invoking 1 Block of Threads.
- **Note**: see `./latexinc/matmul/1/main-sel.cu`

Mat. Mul. (v.2): Grid of 2D Blocks

- `int tx = blockIdx.x*blockDim.x + threadIdx.x;`
- `int ty = blockIdx.y*blockDim.y + threadIdx.y;`



Matrix Mul.: visualization (v. 2)



Matrix Mul.: kernel (v. 2)

Note: see `./latexinc/matmul/2/mul.cu`

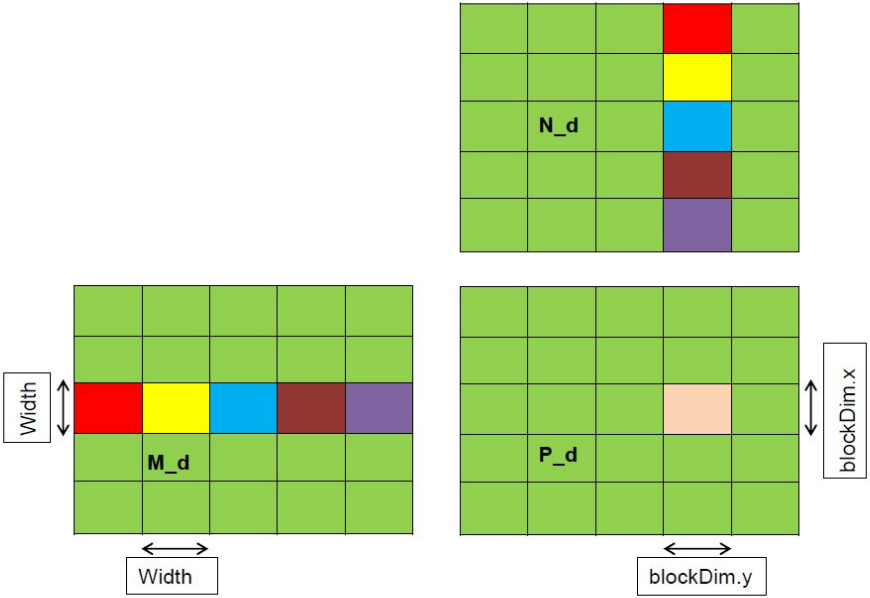
Invoking kernel (v. 2)

- Invoking a grid of blocks of threads.
- **Note**: see `./latexinc/matmul/2/main-sel.cu`

Types of GPU memory

- **global memory**: largest, slowest, often the bottleneck.
- **constant memory**: cached, read-only. `__constant__`: constant memory space specifier.
- **registers**: fast, on-chip memory (exclusive to each thread).
- **shared memory**: allocated per thread block & low latency. `__shared__`: shared memory space specifier. `__syncthreads()`: barrier function which forces all threads in a block to wait until all threads have arrived before proceeding (block level synchronization).

Matrix Mul.: use of shared memory (v. 3)



Matrix Mul.: kernel (v. 3) — use of shared memory

Note: see `./latexinc/matmul/3/mul.cu`

Building/Compiling CUDA applications

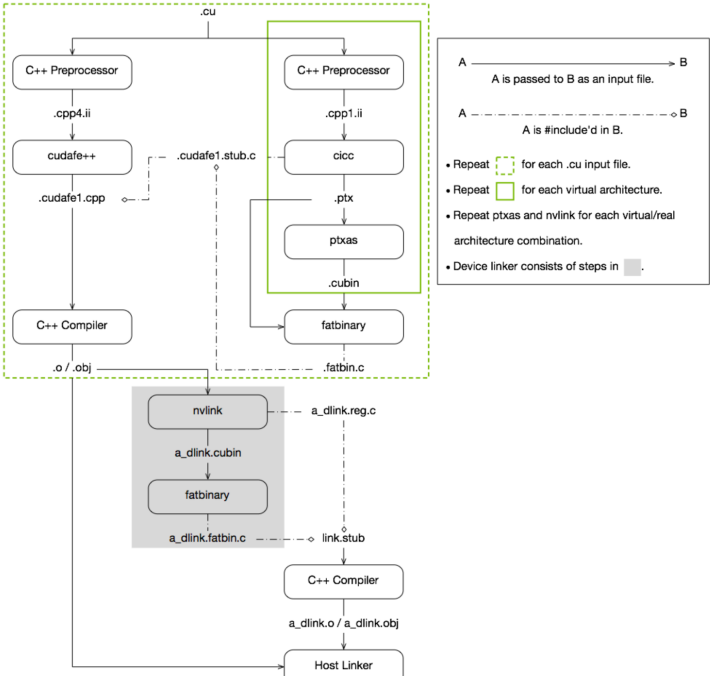
General scheme:

- Source code for CUDA applications: `C`/C++ host code with extensions to deal with the device(s). Other programming languages are allowed e.g. `Fortran`.
- Primo: **separate** device functions from host code.
- **Device code**: preprocessing, compilation with the NVIDIA compiler (`nvcc`).
- **Host code**: preprocessed, compiled with a host (`C`/C++) compiler e. g. (`gcc, g++, icc, icpc, …`)
- Compiled device functions are **embedded** as `fatbinary` images in the host object file.
- **Linking stage**: adding CUDA runtime libraries to the host object file to create an executable.

Further concepts

- `.cu`: Suffix for CUDA source file (host code (`C`,C++) & device code).
- `.cuf`: Suffix for CUDA source file (host code (`Fortran`) & device code).
- `.ptx`: Suffix for **P**arallel **T**hread E**x**ecution (`PTX`) files — an intermediate representation similar to assembly for a **virtual GPU architecture** (prefix: `compute_`, e. g. `compute_70`).
- `.cubin`: Suffix for the **CU**DA device **bin**ary file pertaining to a **real GPU architecture** (prefix: `sm_`, e. g. `sm_70`). Memento: `sm` stands for the physical streaming multiprocessor.
- `fatbin`: Multiple `PTX` [& `cubin`] files are merged into a `fatbin` file.

Compilation trajectory



In praxi,

We will now address the following questions:

- What are the recent CUDA architectures?
- How to find the compute capability (CC) of a device?
- How to build an executable for a particular device?
- How to build an executable for multiple architectures?

Recent CUDA Architectures/Generations

- NVIDIA GPU: xy where x: generation (major), y: minor.
- New generation: major improvement in functionality/chip.
- **Binary** compatibility is **NOT** guaranteed among generations.

Architecture	Year	`compute_` (*virtual*)	`sm_` (*real*)
`Maxwell`	2014	50, 52, 53	50, 52, 53
`Pascal`	2016	60, 61, 62	60, 61, 62
`Volta`	2017	70, 72	70, 72
`Turing`	2018	75	75
`Ampere`	2020	80, 86, 87	80, 86, 87
`Ada Lovelace`	2022	89	89
`Hopper`	2022	90, 90a	90, 90a
`Blackwell`	2024	100, 100a	100, 100a

Some of the recent CUDA architectures (10/31/2025)

Retrieval of the Compute Capability (CC)

You can:

1. Use `nvidia-smi` to display the architecture. `nvidia-smi` is an **extremely powerful** tool to query the state & specs of the devices attached to a node.
2. Use `nvaccelinfo` (part of NVIDIA HPC SDK).
3. Write basic `C`/C++ code relying on CUDA APIs (available in `src/devicequery`, executable: `devinfo`): `cudaGetDeviceCount(int *tot)`: returns number of devices. `cudaGetDeviceProperties(cudaDeviceProp *p, int idev)`: returns information about device `idev`.

Use of nvidia-smi

```
[u0253283@notch001:~]$ nvidia-smi -l
Tue Oct 22 12:38:52 2024
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla V100-PCIE-16GB           On  |   00000000:3B:00.0 Off |                    Off |
| N/A   31C    P0             24W /  250W |       0MiB /  16384MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+


+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
^C[u0253283@notch001:~]$ nvidia-smi --query-gpu=compute_cap --format=csv
compute_cap
7.0
```

nvidia-smi

Retrieval of CC using `nvaccelinfo`

```
[u0253283@notch001:~]$ module load nvhpc
[u0253283@notch001:~]$ nvaccelinfo

CUDA Driver Version:           12040
NVRM version:                  NVIDIA UNIX x86_64 Kernel Module  550.54.14  Thu Feb 22 01:44:30 UTC 2024

Device Number:                 0
Device Name:                   Tesla V100-PCIE-16GB
Device Revision Number:        7.0
Global Memory Size:            16928342016
Number of Multiprocessors:     80
Concurrent Copy and Execution: Yes
Total Constant Memory:         65536
Total Shared Memory per Block: 49152
Registers per Block:           65536
Warp Size:                     32
Maximum Threads per Block:     1024
Maximum Block Dimensions:      1024, 1024, 64
Maximum Grid Dimensions:       2147483647 x 65535 x 65535
Maximum Memory Pitch:          2147483647B
Texture Alignment:             512B
Clock Rate:                    1380 MHz
Execution Timeout:             No
Integrated Device:             No
Can Map Host Memory:           Yes
Compute Mode:                  default
Concurrent Kernels:            Yes
ECC Enabled:                   No
Memory Clock Rate:             877 MHz
Memory Bus Width:              4096 bits
L2 Cache Size:                 6291456 bytes
Max Threads Per SMP:           2048
Async Engines:                 7
Unified Addressing:            Yes
Managed Memory:                Yes
Concurrent Managed Memory:     Yes
Preemption Supported:          Yes
Cooperative Launch:            Yes
  Multi-Device:                Yes
Default Target:                cc70
```

Retrieval of CC through some simple CUDA APIs

```
[u0253283@notch001:~]$ devinfo
Node: notch001
#Devices detected: 3


  Device:0
    Device Name : Tesla V100-PCIE-16GB
    ECC Enabled              : 0
    Compute Capability       : 70
    Compute Mode             : 0
    #SM on device            : 80
    Device Clock Rate (GHz) : 1.3800E+00
    Peak Memory clock frequency (GHz): 8.7700E-01
    L2 Cache Size (bytes)            : 6291456
    Warp Size in Threads             : 32

    Max. #Blocks/SM                  : 32
    Max. Size of each dim. of a Grid : (2147483647,65535,65535)
    Max. Size of each dim. of a Block: (1024,1024,64)
    Max. #Threads/Block              : 1024
    Max. Resident Threads/SM         : 2048
    Global Mem. available on device (bytes)  : 16928342016
    Constant Mem. available on device (bytes): 65536
    #32-bit Registers available per SM       : 65536
    #32-bit Registers available per Block    : 65536
    Shared Mem. available per Block (bytes)  : 49152
    Shared Mem. available per SM (bytes)     : 98304
```

Compiling your code for a particular device

- Compilation goes in 2 steps:
- Step 1 — `PTX` representation: generic assembly for a *virtual* (`compute_`) GPU architecture. The `.ptx` file is human readable (text file).
- Step 2 — `Binary generation`: object file for the *real* (`sm_`) GPU architecture (based on the `PTX` file).
- `-arch`/`-code` flags:
- `--gpu-architecture|-arch <arch>`: specifies 1 *virtual* GPU architecture, e. g. `-arch=compute_50`.
- `--gpu-code|-code <arch>`: specifies the *real* GPU architecture(s), e. g. `-code=sm_52`.

Compiling your code (cont.)

- Choose 1 virtual architecture and the accompanying real architectures, e. g. `-arch=compute_50 -code=sm_50,sm_51,sm_52`: PTX file generated for `compute_50` (*virtual*). **fatbinary** object created for `sm_50,sm_51,sm_52` (*real*).
- `--generate-code|-gencode arch=<arch>,code=<code>`: **Generalization** of `--gpu-architecture=<arch> --gpu-code=<code>`. Allows creation of binaries for different architectures.
- Example: see `./latexinc/cuflags.txt`

Profiling & debugging

CUDA SDK comes with:

- Its own profiler: `nvprof`.
- Its own debugger: `nvsight`.

Profiling `mul3` using nvprof

```
[u0253283@notch001:3]$ nvprof ./mul3
==2424062== NVPROF is profiling process 2424062, command: ./mul3
 Calling Kernel ...
 Kernel Call Finished ...

 Frob. Norm(P-P_h):    0.0000000000

==2424062== Profiling application: ./mul3
==2424062== Profiling result:
            Type  Time(%)      Time     Calls       Avg       Min       Max  Name
 GPU activities:   59.70%  1.1707ms         1  1.1707ms  1.1707ms  1.1707ms  MatrixMulKernel3(double*, double*, double*, int)
                   32.02%  627.87us         2  313.93us  307.13us  320.73us  [CUDA memcpy HtoD]
                    8.28%  162.33us         1  162.33us  162.33us  162.33us  [CUDA memcpy DtoH]
      API calls:   95.82%  138.45ms         3  46.151ms  109.78us  138.23ms  cudaMalloc
                    2.88%  4.1668ms         3  1.3889ms  494.15us  3.1420ms  cudaMemcpy
                    0.82%  1.1901ms         1  1.1901ms  1.1901ms  1.1901ms  cudaLaunchKernel
                    0.27%  395.44us         3  131.81us  94.455us  202.38us  cudaFree
                    0.17%  250.22us       114  2.1940us     291ns  103.89us  cuDeviceGetAttribute
                    0.01%  16.428us         1  16.428us  16.428us  16.428us  cuDeviceGetPCIBusId
                    0.01%  12.713us         1  12.713us  12.713us  12.713us  cuDeviceGetName
                    0.00%  1.5660us         3     522ns     321ns     905ns  cuDeviceGetCount
                    0.00%  1.2730us         2     636ns     338ns     935ns  cuDeviceGet
                    0.00%     613ns         1     613ns     613ns     613ns  cuModuleGetLoadingMode
                    0.00%     556ns         1     556ns     556ns     556ns  cuDeviceTotalMem
                    0.00%     547ns         1     547ns     547ns     547ns  cudaGetLastError
                    0.00%     337ns         1     337ns     337ns     337ns  cuDeviceGetUuid
```

Profiling `mul3` on `notch001`

Important CUDA libraries

To increase performance, use highly-optimized libraries:

- **cuBLAS**: **B**asic **L**inear **A**lgebra **S**ubroutines on NVIDIA GPUs.
- **MAGMA**: **M**atrix **A**lgebra on **G**PU and **M**ulti-core **A**rchitectures.
- **cuRAND**: Random Number Generation library.
- **cuFFT**: CUDA **F**ast **F**ourier **T**ransform library.
- **NCCL**: **N**VIDIA **C**ollective **C**ommunications **L**ibrary.
- **cuDNN**: CUDA **D**eep **N**eural **N**etwork library.
- **cuTENSOR**: GPU-accelerated Tensor Linear Algebra.
- **DALI**: NVIDIA **Da**ta Loading **Li**brary.
- ...

Alternatives to CUDA

- Similar to CUDA: **ROCM** (AMD).
- **OpenACC** (use of directives, cfr. `OpenMP`): GCC supports OpenACC for NVIDIA & AMD GPUs. NVIDIA HPC SDK (formerly PGI). Sourcery Codebench (AMD GPU).
- Higher-level abstractions: **Kokkos** (programming model for parallel algorithms on many-core chips).

Links

- CUDA Toolkit Documentation
- CUDA C++ Programming Guide Release 13.0 (10/31/25)
- CUDA C++ Best Practices Guide, Release 13.0 (10/31/25)
- NVIDIA CUDA Compiler Driver NVCC, Release 13.0 (10/31/25)
- PTX & ISA Release 9.0 (10/31/25)

GPU devices on lp/kp/np/grn

GPU device type	CC
NVIDIA GeForce GTX TITAN X	5.2
Tesla P100-PCIE-16GB	6.0
Tesla P40	6.1
NVIDIA GeForce GTX 1080 Ti	6.1
NVIDIA Titan V	7.0
NVIDIA Tesla V100-PCIE-16GB	7.0
Tesla T4	7.5
NVIDIA GeForce RTX 2080 Ti	7.5
NVIDIA A100-PCIe-40GB	8.0
NVIDIA A100-SXM4-80GB	8.0
NVIDIA A800 40GB Active	8.0

GPU devices on lp/kp/np/grn (10/31/2025). CC = compute capability.

GPU devices on lp/kp/np/grn (cont.)

GPU device type	CC
NVIDIA GeForce RTX 3090	8.6
NVIDIA A40	8.6
NVIDIA RTX A5500	8.6
NVIDIA RTX A6000	8.6
NVIDIA RTX 6000 Ada Generation	8.9
NVIDIA L40	8.9
NVIDIA L40S	8.9
NVIDIA H100 [NVL]	9.0
NVIDIA H200 [NVL]	9.0
NVIDIA RTX PRO 6000 Blackwell Max-Q	12.0

GPU devices on lp/kp/np/grn (10/31/2025). CC = compute capability.

GPU devices on redwood

GPU device type	CC
NVIDIA GeForce GTX 1080 Ti	6.1
NVIDIA A100-SXM4-40GB	8.0
NVIDIA A100 80GB PCIe	8.0
NVIDIA A30	8.0
NVIDIA A40	8.6
NVIDIA RTX 6000 Ada Generation	8.9
NVIDIA H100 NVL	9.0
NVIDIA H200 [NVL]	9.0

GPU devices on redwood (10/31/2025). CC = compute capability.

Accessing GPUs at CHPC

- Using GPUs at the CHPC (Presentation by Martin Čuma)
- **Note:** When a GPU job is launched it runs with its own cgroup (limits/accounts for its own resources).
- When a $USER has several GPU jobs running on the **same** node, the $USER will land in **one** cgroup belonging to one of their jobs when `ssh`ing into that node. Therefore, the $USER **cannot** verify the status of other jobs using `nvidia-smi` directly.
- ⇒ Use instead:
  `srun --pty --overlap --jobid $JOBID /usr/bin/nvidia-smi`
  where `JOBID` is the job identifier.

Questions?

Thank you!
Any questions?